

2.1.2 Test Quality and Coverage The quality of your tests and assertions will be assessed as well so pay attention to the quality of your test code. Discuss the measures you took for ensuring the quality of your test cases in your report. Code coverage has been traditionally used as a popular test adequacy criterion. Measure, report, and discuss line and branch coverage of your tests. Document and explain the results in your report. Discuss whether there any features or code segments that are not covered and why.

Our overall results using Jacoco we got 68% instruction coverage and 76% branch coverage. These results combine both the fully tested gameplay logic and the intentionally untested JavaFX UI visuals. Most of the remaining uncovered code is in the GameHandler, UiHandle, Main, and DrawEntity classes. These classes mostly have rendering, animations, and scene setups.

Some measures we took to ensure the quality of our test cases is that for the tests that use randomness (like gems spawning randomly), we used FakeRandom, which extends the Random class to insert objects at the precise location where we want them to ensure the test results are accurate. We also used isolated unit tests to make sure all branches were covered. Lastly, I would like to add one big reason most of the tests did not reveal any bugs is because we were actively making sure the code pushed was working to the best of our knowledge.

2.1.3 Findings Briefly discuss what you have learned from writing and running your tests. Did you make any changes to the production code during the testing phase? Were you able to reveal and fix any bugs and/or improve the quality of your code in general? Discuss your more important findings in the report.

1. **MonsterHTest:** Here I was trying to implement a test to make sure the handleMonster function was doing its job by ending the round if the player and monster were at the same coordinates. I spent almost the entire day (on a very minor issue) and changed a few small things to improve the quality of the code such as a break statement instead of a return, which wouldn't advance the code to check collisions in certain rare situations. The most important part that was causing the test to keep on failing was that there was no initial check for collisions on spawn, which I added in the handleMonster functionality.

2. **GameObjectTest:** Most tests went according to plan here, however when implementing test methods for the trap gameObject, it would fail due to spawning the trap on top of other entities when the trap was moving after being hit. After further review, in trap.interact when moving the trap, it only checked for the new tile being a wall, not another entity.
3. **ChestHandlerTest:** When testing the chestHandler.update method, there were issues with checking whether or not chests had been removed after being collected or timing out. This was because in chestHandler.update the chest was removed from the local list of chests, so there weren't any problems with the flow of the method, but the list being passed as a parameter did not change. This was an easy fix, just had to remove the collected/timed out chest from the passed parameter list.
4. **ExitHandlerTest:** When testing exitHandler i decided the exit should probably not be randomized, since that could make the level a lot harder/easier than it should be, so i added spawn points for exit and SpawnBounds and made ExitHandler follow them.
5. **CollisionHandlerTest:** To implement the collision handler test I simulated the instantiation of the map and positioned player and monster objects within it to mainly test the raycast feature used by the monster class to implement vision. When writing the tests I started out with a basic raycast invocation to test a random scenario. Reviewing the code I have written, I learned that there are certain branches in the algorithm that are never reached, so I created new tests in order to reach them. During testing I found out that my code does not handle null arguments, so in production code, I added a null check that throws an exception. Aside from the small change, my tests did not improve the quality of the code.
6. **PathfinderTest:** My approach to testing the monster's pathfinding algorithm was to spawn in monster and player mock objects randomly across the map and invoke the pathfinding algorithm. While testing the pathfinding algorithm, I noticed that the monster class instantiates a pathfinding object every time it needs to create a path to the player. Object instantiation is an expensive task and creating a scenario with thousands of monsters could downgrade performance exponentially. To

fix this, I utilized the singleton design pattern and instantiated one pathfinding object per monster. I then refactored the pathfinding class using an imperative approach, creating methods that change the state of the object. Intuitively speaking, this should increase the performance immensely because we only construct the pathfinding object once per monster instantiation and java does not need to automatically garbage collect objects without a reference to them. Because of time constraints, what I would like to do differently is to take the optimization a step further and have one pathfinding instance being shared by all monster objects, this would work because the program is single threaded so race conditions wouldn't be possible.

7. **MonsterTest:** To test the monster's random movement behaviour during its roaming state, I used mock monster objects and placed them in random spots to test what its available directions are. These tests did not reveal any bugs.
8. **MapTest:** This tested the constructor initialization, tile boundaries, wall placement, isWall logic, and getTile behavior and tested key samples from createLevel1(). All tests passed successfully with no issues or bugs found.
9. **RoundTest:** Unit tests that exercised round start/end, entityGrid operations, and default initialization of lists and map. All tests passed successfully with no issues or bugs found.
10. **GameHandlerTest:** Used reflection to test private pause/resume/toggle logic. All tests passed successfully with no issues or bugs found.

Integration tests:

1. **playerCollectsGems_thenHitsTrap_thenWinsExit():** This test tests the gameflow, making sure player interacting with multiple different items results in the correct outcome.
2. **playerCollectsGem_thenHitsTrapAndDiesLol():** This test also tests gameflow, making sure player loses when running into a trap resulting in negative score, even after collecting a gem first.
3. **constructor_initializesRoundAndSpawnsEntities():** This test makes sure the gameHandler interacts properly with all the

gameObjectHandlers. It asserts that the correct number of each entity has been created, as well as the round being initialized, and the player being in the correct spot.

4. **PathFinderTest:** This test class focuses on testing the interactions between the pathfinder class, the monster class, and the player class. This is used to make sure the monster is getting the correct path to the player so it can chase the player.
5. **constructor_initializesCollectionsAndMap():** This test makes sure the round class initializes all entities correctly.